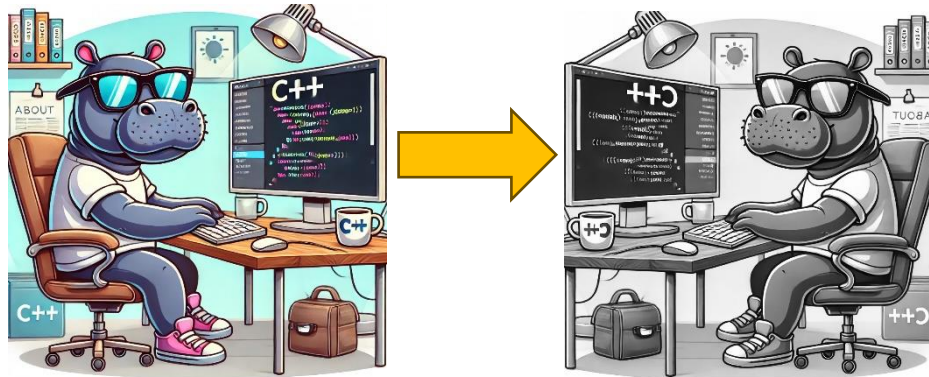


# CSC 1310 LAB 7

## IMAGE PROCESSING WITH QUEUE & TASK PARALLELISM SPRING 2025



### SKILLS

- Prerequisite Skills Needed:
  - Basic C++ (arrays, structures, arrays of structures, pointers, ifstream/ofstream)
  - **Queue**
- Parallel Computing Concepts Learned in this Lab: **Pipelining, Task Parallel**

### ASSIGNMENT OBJECTIVE

- You will be able to implement an algorithmic solution that uses task parallelism.
- You will store data using the queue data structure in your program.

### DESCRIPTION OF THIS LAB

For this lab, you will be implementing filters to grayscale and flip an image (in ppm format). You will then modify your code so that it processes the filters in parallel via a pipeline, thus speeding up the applications of the functions. This lab will first give you background information on applying filters via a pipeline and task parallelism.

### YOU MAY PARTNER UP!

You may either work alone or partner up with ONE other person in YOUR LAB SECTION on this lab.

**This lab is best done on a Windows computer.** Technically, it can work on a Mac by doing an extra step of running a shell script. Both partners must submit the same code to ilearn.

**If you have a Mac, you should partner up with someone with Windows and work off their computer.**

However, if you really want to be able to run the code on a Mac, there is an additional file included in Lab 7 in ilearn called **Mac-Only Directions** that you should download.

## BACKGROUND INFORMATION

### MANIPULATING IMAGES



Images are represented as **pixels**. You can think of a pixel and an individual color "dot" on your monitor screen.

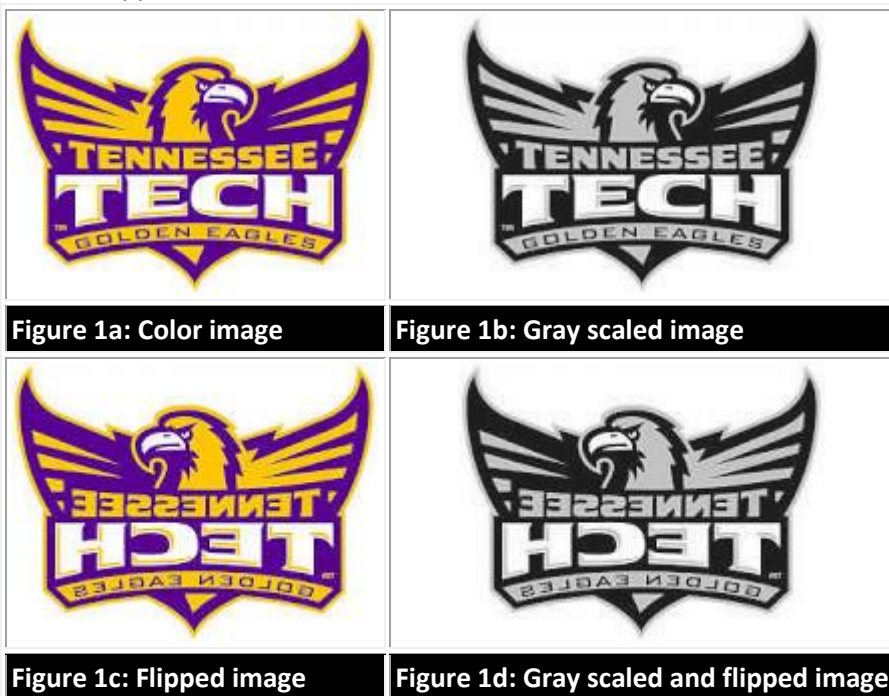
The color of the pixel is represented as a mixture of intensities of the colors **red**, **green**, and **blue**.

Each intensity is represented as an 8-bit number in the ranging from 0 to 255. For example, the values (0,0,0) represent the color **black**, the values (255,0,0) represent **red**, and the values (255,255,0) represent **yellow**.

We call these intensities **RGB values** (R=red, G=green, B=blue).

|                                           |                                           |                                          |                                          |
|-------------------------------------------|-------------------------------------------|------------------------------------------|------------------------------------------|
| Lime<br>#A4C400<br>RGB(164,<br>196, 0)    | Green<br>#60A917<br>RGB(96,<br>169, 23)   | Emerald<br>#008A00<br>RGB(0,<br>138, 0)  | Teal<br>#00ABA9<br>RGB(0,<br>171, 169)   |
| Cyan<br>#1BA1E2<br>RGB(27,<br>161, 226)   | Cobalt<br>#0050EF<br>RGB(0, 80,<br>239)   | Indigo<br>#6A00FF<br>RGB(106,<br>0, 255) | Violet<br>#AA00FF<br>RGB(170,<br>0, 255) |
| Pink<br>#F472D0<br>RGB(244,<br>114, 208)  | Magenta<br>#D80073<br>RGB(216,<br>0, 115) | Crimson<br>#A20025<br>RGB(162,<br>0, 37) | Red<br>#E51400<br>RGB(229,<br>20, 0)     |
| Orange<br>#FA6900<br>RGB(250,<br>104, 0)  | Amber<br>#F0A30A<br>RGB(240,<br>163, 10)  | Yellow<br>#F5C000<br>RGB(227,<br>200, 0) | Brown<br>#825A2C<br>RGB(130,<br>90, 44)  |
| Olive<br>#6D8764<br>RGB(109,<br>135, 100) | Steel<br>#647687<br>RGB(100,<br>118, 135) | Mauve<br>#76608A<br>RGB(118,<br>96, 138) | Taupe<br>#87794E<br>RGB(135,<br>121, 78) |

In this lab, you will be writing a program to manipulate images. The two image processing techniques that you will implement are **gray-scaling** and **flipping an image**. Below is an example of an image that has been gray-scaled, flipped, and then both gray-scaled and flipped.



### GRAY SCALING AN IMAGE

Gray scaling an image represented as a series of RGB values is easy. Different methods exist, but an effective method is called the luminosity method. Given the  $i$ th pixel, you can grayscale that pixel with the following formula:

$$\text{GRAY\_VALUE}[i] = 0.21 * \text{PIXEL}[i].\text{RED} + 0.72 * \text{PIXEL}[i].\text{GREEN} + 0.07 * \text{PIXEL}[i].\text{BLUE}$$

Then, for each  $i$ , set the red, green, and blue component of  $\text{PIXEL}[i]$  to  $\text{GRAY\_VALUE}[i]$  to grayscale the image.

## FLIPPING AN IMAGE HORIZONTALLY

Flipping the image is also easy. Flip an image by flipping the 1st pixel with the last pixel, the 2nd pixel with the next-to-last pixel, the 3rd pixel with the second-to-last pixel, and so on.

## PPM IMAGE FILES

A **PPM** (Portable Pixmap Format) image file is a simple raster image format used for storing color images in an uncompressed, human-readable text format (ASCII) or a binary format. We will be using these types of image files in this lab because of their simplicity of opening/closing and reading/writing in binary format, ease of parsing, and lack of compression. PPM files do not require advanced decoding like JPEG, PNG, or GIF files and so manipulating the images with code is easier and straightforward.

A PPM file consists of:

- a small header (format, dimensions, and color depth)
- Raw pixel data (RGB values)

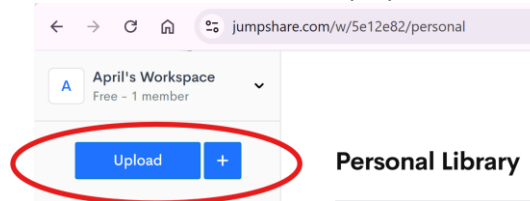
This makes it easy to read and write in a basic C++ program without using third-party libraries.

The downside? You can't just simply double click on the image file to open it.

**Instead, you must use a free, online PPM reader:** <https://jumpshare.com/file-sharing/ppm>

**You will have to create a free account to be able to view large PPM files for this lab. However, you do not have to install anything – you can just use the web online viewer.**

So, when you want to view any PPM image, just go to this website, log in, and then click the upload button to upload the PPM file on your computer to the website. It will convert it and display it on the screen.

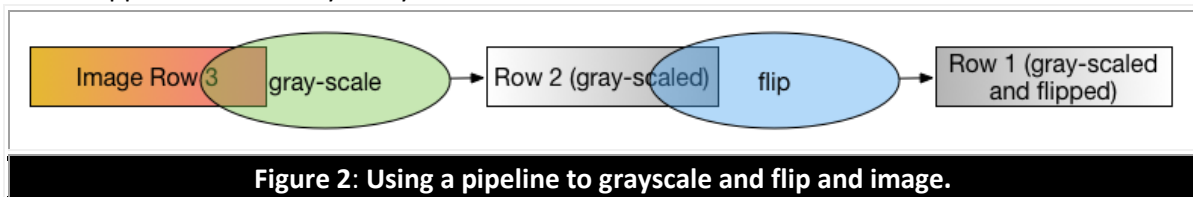


## PIPELINING AND TASK PARALLELISM

You will be using an image processing technique called **filtering**. Filtering is simply applying a sequence of operations to an image to achieve a desired outcome. For example, if you wanted to show an image as if it were depicted as shown in a mirror on an old black-and-white TV, you would apply the grayscale and flip filters. Fortunately, for many image filters, such as grayscale and flipping, you can accomplish the filtering in parallel and therefore reduce the computation time.

One method of filtering is called **pipelining**, which is a form of functional decomposition. **Functional decomposition** in parallel programming is like breaking a big job into smaller independent **tasks** (or **functions**) that can be done at the same time by different workers.

As shown in **Figure 2** below, **pipelining** is a form of **functional decomposition** in which a series of filters, or functions, manipulates a portion of the input data, and then passes the data onto the next function, which begins processing the data. **Pipelining** is like an assembly line in a factory. Instead of one worker doing everything, each worker specializes in a part of the process. The work flows through different stages, with each stage running in parallel. Note that the technique can be applied to arbitrarily many functions.



Notice that in **Figure 2**, the rectangles are overlapping with the ovals to represent that the gray-scale function is simultaneously processing image row 3 while flip is processing row 2, which was previously processed by grayscale.

So, how do you implement a pipeline. Consider **Figure 3**. A straightforward way to implement pipelining is to use a **queue** to pass data between the functions. When the first function, which is **gray-scale()** in this example, finishes a row, it passes that row to flip by putting the row in a shared queue. When the **flip()** function is ready to process the next row, it checks the queue. If the queue has a row in it, then the flip function removes the row and processes it.

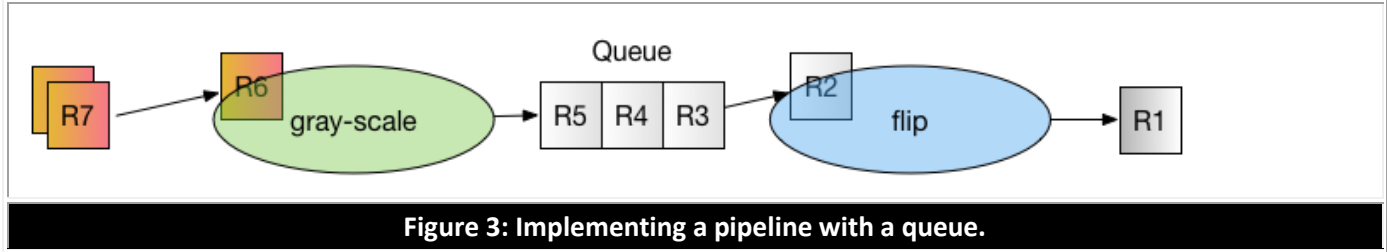


Figure 3: Implementing a pipeline with a queue.

## INSTRUCTIONS

### REQUIRED SETUP

1. Create a **Lab7** folder. Download the given zip file from Lab 7 in ilearn and put them in your **Lab7** folder:
  - a. **libppm.cpp**
  - b. **libppm.h**
  - c. **ttu.ppm**
  - d. **ttu\_tile.ppm**
2. **VIEW the given image:** use the website mentioned above (<https://jumpshare.com/file-sharing/ppm>) to upload the **ttu.ppm** file so you can see the image.
3. Create a text document to record information you get from this lab called **lab7.txt**. You will be instructed when to add information to this file.

### PART 1 – CREATE THE DRIVER

1. Create **lab7.cpp** in your **Lab7** folder.
2. Put the following code in **lab7.cpp**:

```
#include <iostream>
#include <fstream>
#include <stdexcept>
#include <sstream>
#include "libppm.h"
using namespace std;

int main()
{
    string ppmInFile, ppmOutFile;
    PPM_header img_header;
    ifstream inFile;
    ofstream outFile;
    RGB_8 *img; //a pointer to a RGB_8 structure

    cout << "\n\nInput Image Name (in_image.ppm): ";
    cin >> ppmInFile;
```

```

cout << "Output Image Name (out_image.ppm): ";
cin >> ppmOutFile;

//open the input image file in binary mode
inFile.open(ppmInFile, ios::binary);
if(!inFile.is_open()) {
    cout << "Unable to open file: " << ppmInFile<< endl;
    return 1;
}

//read the file and store in program
PPM_read_header(inFile, img_header);
cout << img_header << endl;
img = new RGB_8[img_header.height * img_header.width];
PPM_read_rgb_8(inFile, img_header.width, img_header.height, (RGB_8 *)img);

//open the output file in binary mode
outFile.open(ppmOutFile, ios::binary);
if(!outFile.is_open()){
    cout << "Unable to open file: " << ppmOutFile << endl;
    return 1;
}

//grayscale the image
toGrayScale(img, img_header.height, img_header.width);

//flip the image
flip(img, img_header.height, img_header.width);

//write the updated image to the output file
PPM_write_header_8(outFile, img_header.width, img_header.height);
PPM_write_rgb_8(outFile, img_header.width, img_header.height, (RGB_8 *)img);

//close both files
inFile.close();
outFile.close();

return 0;
}

```

3. Compile & run the code using the sample output below. User input is highlighted in yellow.

```

Input Image Name (in_image.ppm): ttu.ppm
Output Image Name (out_image.ppm): ttu-out.ppm
183 140 255

```

4. Make sure that your code works by viewing **ttu-out.ppm** at <https://jumpshare.com/file-sharing/ppm> . It should look like the image below:



## PART 2 – TEST WITH LARGE PPM FILE

1. You are now going to time your code running. Add the C header file `sys/time.h`, which can measure time intervals with high precision.

```
#include <sys/time.h>
```

2. Add the following code immediately before the call the `toGrayScale()` function in `lab7.cpp`:

```
struct timeval tv1;  
struct timeval tv2;  
gettimeofday(&tv1, NULL);
```

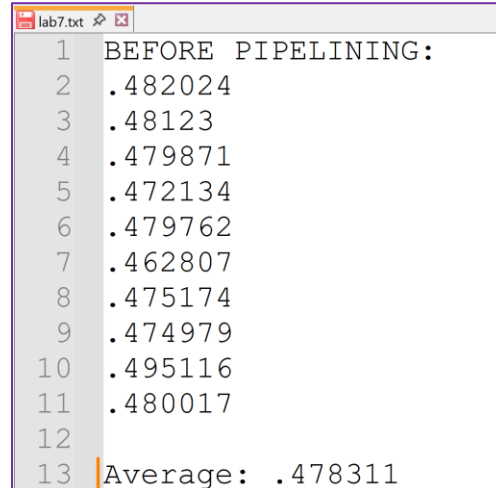
3. Next, add the following code immediately after you call the `flip()` function in `lab7.cpp`:

```
gettimeofday(&tv2, NULL);  
cout << "Total time: ";  
cout << static_cast<double>(tv2.tv_usec - tv1.tv_usec) / 1000000 +  
        static_cast<double>(tv2.tv_sec - tv1.tv_sec) << endl;
```

4. In your `lab7.txt` text file, type on the first line “BEFORE PIPELINING:”.
5. Now, run your code **TEN** times with your input file as `ttu_tile.ppm` and output file as `ttu_tile_out.ppm` and record the time in your `lab7.txt` text file as you see below.

Then, calculate the **average** time and put that in the text file as well as you see below.

(Note that your times will likely be a little different because the time varies based on what you are running in the background as well as the hardware on your computer.)



```
1 BEFORE PIPELINING:  
2 .482024  
3 .48123  
4 .479871  
5 .472134  
6 .479762  
7 .462807  
8 .475174  
9 .474979  
10 .495116  
11 .480017  
12  
13 Average: .478311
```

## PART 3 – ADD PIPELINING

Now, you are going to modify your code so the image filters will be applied in a **pipeline**. Note that once a row of pixels is finished being gray scaled, the row can be immediately flipped. Time can be saved by flipping an already gray scaled row, and, at the same time, gray scaling the next row in the image. Unfortunately, you must modify the code to take advantage of this parallelism. Fortunately, you have some help. OpenMP is a tool that simplifies taking advantage of parallelism.

You will modify your code so that the `toGrayScale()` function enqueues (adds to the end) the row that it just finished gray scaling. Then, you will modify your `flip()` routine by removing the outer for loop that loops through the rows and replacing it with a call to dequeue (remove from the front) the next row to flip. Using a **queue** in this way



implements a pipeline between the two functions. We will be using the **Queue** from the C++ STL. However, we must create our own **enqueue ()** and **dequeue ()** functions to make pipelining work.

1. First, `#include<queue>` at the top of the `libppm.h` header file.
2. Under the structure declarations in `libppm.h`, write the following statement:  
`extern queue<RGB_8 *> pipeline;`

The *extern* keyword in C++ is used to declare a variable or function that is defined in another source file. It tells the compiler not to allocate storage for the variable or function, but instead to look for its definition elsewhere.

3. Declare your **queue** at the top of your `lab7.cpp` file globally. It should be directly under **using namespace std**:  
`queue<RGB_8 *> pipeline;`
4. Add the following code (enqueue and dequeue functions) to the `libppm.cpp` source file.

This code accomplishes two goals.

First the queue is a shared resource between the `toGrayScale ()` function and the `flip ()` function, so it must be protected from concurrent access using the `#pragma omp critical (pipeline)` OpenMP directive.

Second, the `flip ()` function can only flip a row when a row is available.

```
RGB_8 *dequeue(std::queue<RGB_8 *> &q) {
    RGB_8 *image_row;
    bool keep_checking = true;
    while (keep_checking) {
        #pragma omp critical (pipeline)
        {
            if (!pipeline.empty()) {
                image_row = pipeline.front();
                pipeline.pop();
                keep_checking = false;
            }
        }
        #pragma omp taskyield
    }
    return image_row;
}

void enqueue(std::queue<RGB_8 *> &q, RGB_8 *row) {
    #pragma omp critical (pipeline)
    {
        pipeline.push(row);
    }
}
```

Also, add the function prototypes to both `dequeue ()` and `enqueue ()` functions in `libppm.h` along with the other function prototypes.

5. Next, modify your `toGrayScale()` code such that, at the end of processing each row, that row is enqueued.

```
void toGrayScale(IMG_8 *img, int height, int width) {
    int index;
    unsigned char grayval;

    for (int row = 0; row < height; row++) {
        IMG_8 *row_data = &img[row * width]; //Use image memory directly
        for (int col = 0; col < width; col++) {
            index = row * width + col;
            grayval = static_cast<unsigned char>(
                0.21 * static_cast<unsigned char>(img[index].r) +
                0.72 * static_cast<unsigned char>(img[index].g) +
                0.07 * static_cast<unsigned char>(img[index].b));
            img[index].r = grayval;
            img[index].g = grayval;
            img[index].b = grayval;
        }
        //Store existing row instead of allocating new memory
        enqueue(pipeline, row_data);
    }
    enqueue(pipeline, NULL); //Signal flip() that processing is done
}
```

Note the `enqueue(pipeline, NULL)` at the very end. The NULL value is used to signal the `flip()` function that no more rows are available.

6. Next, modify the `flip()` function. Following is the code you need to add the dequeue function call:

```
void flip(IMG_8 *img, int height, int width) {
    IMG_8 *row;
    for (int row_idx = 0; row_idx < height; row_idx++) {
        do {
            row = dequeue(pipeline); // Get a row from the queue
        } while (row == nullptr); // Keep checking if the queue is empty

        for (int col = 0; col < width / 2; col++) {
            swap(row[col], row[width - col - 1]);
        }

        //Store the flipped row back into img
        memcpy(&img[row_idx * width], row, width * sizeof(IMG_8));
    }
}
```

7. Compile your program. Add the "-fopenmp" option to your compiler command:

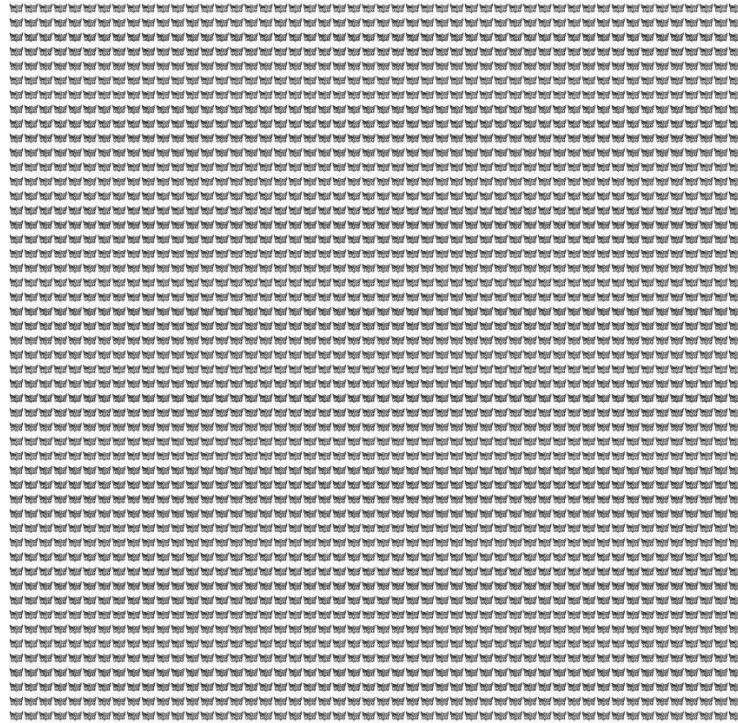
```
g++ -fopenmp -o -Wall lab7.cpp libppm.cpp
```

`-fopenmp` is a compiler flag used in GCC (GNU Compiler Collection) and Clang to enable OpenMP (Open Multi-Processing) support in C, C++, and Fortran programs. When you compile a program with `-fopenmp`, it does the following:

- Enables OpenMP directives (`#pragma omp ...`), allowing the compiler to recognize OpenMP parallelization commands
- Links the OpenMP library (libgomp for GCC) to support multi-threaded execution.
- Generates parallelized code based on the OpenMP constructs in your program.

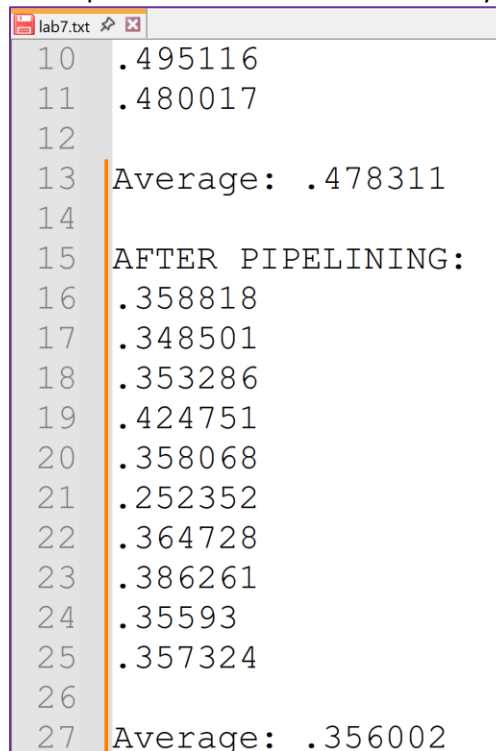


8. Test it to make sure that it works by first using **ttu.ppm** as the input file and **ttu\_out.ppm** and then checking to ensure **ttu\_out.ppm** is both gray scaled and flipped. Mine is below. You would have to zoom in to see that the image is flipped.



6. Add a new line to your **lab7.txt** text file that says “AFTER PIPELINING:”.
7. Now, run your code **TEN** times with your input file as **ttu\_tile.ppm** and output file as **ttu\_tile\_out.ppm** and record the time in your **lab7.txt** text file as you see below.

Then, calculate the **average** time and put that in the text file as well as you see below.



```
lab7.txt
10 .495116
11 .480017
12
13 Average: .478311
14
15 AFTER PIPELINING:
16 .358818
17 .348501
18 .353286
19 .424751
20 .358068
21 .252352
22 .364728
23 .386261
24 .35593
25 .357324
26
27 Average: .356002
```

8. Last, calculate the **speedup** of the parallel version (subtract the after-pipeline time from the before pipeline time) and record the speedup in **lab7.txt**. **Is the parallel code faster?**

## FILL OUT THE LAB REPORT

You will fill out this lab report for every lab and it is part of your grade. To get credit, you must upload a screenshot of the confirmation page to this lab assignment. Name your screenshot **lab7ReportProof**.

**Lab Report Link:** [https://tntech.co1.qualtrics.com/jfe/form/SV\\_8CgQBOeBBefzeCO](https://tntech.co1.qualtrics.com/jfe/form/SV_8CgQBOeBBefzeCO)

## WHAT TO TURN IN

Create a zip file named **TTUusername\_lab7.zip** containing the following files and upload it to ilearn. Replace TTUusername your Tennessee Tech username. Example: **jdoe43\_lab7.zip**

- **libppm.h**
- **libppm.cpp**
- **lab7.cpp**
- **lab7.txt**
- **ttu.ppm**
- **ttu\_out.ppm**
- **ttu\_tile.ppm**
- **ttu\_tile\_out.ppm**
- **Lab7ReportProof**